

Learning Mixed Strategies in Zero-Sum LQ Games via Model-Free Actor-Critic Framework

Technical Report: Problem Formulation, Network Architecture, and Training Pipeline

Working Draft — March 17, 2026

Contents

1	Introduction and Motivation	2
2	The Zero-Sum LQ Game: Full Problem Statement	2
2.1	General Formulation	2
2.2	One-Dimensional Pursuit-Evasion Game (Benchmark)	3
2.3	Analytic Equilibrium under Mixed Strategies ($Q = 0$)	3
2.4	Non-Uniqueness of Equilibrium Strategies ($Q = 0$)	4
2.5	The SDG Approximation Result	5
3	Model-Free Actor-Critic Framework	6
3.1	High-Level Architecture	6
3.2	Q-Function Semantics	6
4	Network Architectures	7
4.1	Actor Network	7
4.2	Critic Network (Double Q)	7
5	Training Pipeline	8
5.1	Overview	8
5.2	Replay Buffer	9
5.3	Critic Update	9
5.4	Actor Update	10
5.5	Evaluation	11
5.6	Hyperparameter Summary	11
6	Current Results and Observations	11
6.1	Training Trajectory ($Q = 0$, 5000 episodes)	11
6.2	Planned Fix: Adding Running Cost $Q > 0$	12
7	Summary of Key Bug Fixes and Design Decisions	12
8	Next Steps	12

1 Introduction and Motivation

This report provides a self-contained and rigorous account of our work on learning Nash equilibrium (NE) mixed strategies for zero-sum linear-quadratic (LQ) games in a **model-free** setting. The work extends the results of “*Learning Mixed Strategies in Zero-Sum Linear Quadratic Games*” (NeurIPS 2024 submission), which operates under a **known-model** assumption.

Original paper’s contribution. The original paper makes two key contributions:

- (i) It proves that the optimization landscape of zero-sum LQ games under state-feedback mixed strategies can be reduced to the **first two conditional moments** of the control variables—the conditional mean $\Gamma_i(k, x)$ and the conditional covariance $\Sigma_i(k, x)$. This is established via the method of *Stochastic Differential Game (SDG) approximation*.
- (ii) It designs *stacked randomized policy networks* with noise inputs to learn equilibrium mixed strategies, leveraging full knowledge of the system dynamics (A, B_1, B_2) for gradient back-propagation through the linear dynamics layer.

Our extension. We remove the known-model assumption and replace the dynamics-layer back-propagation with an **Actor-Critic (AC)** framework. The Critic receives the opponent’s moment information $(\Gamma_{-i}, \Sigma_{-i})$ rather than the opponent’s full action distribution, directly operationalizing the SDG dimension-reduction result.

2 The Zero-Sum LQ Game: Full Problem Statement

2.1 General Formulation

Consider a two-player zero-sum LQ game on a finite horizon $N \in \mathbb{N}$.

Definition 2.1 (Game Dynamics). *The state evolves according to the linear system*

$$x(k+1) = Ax(k) + B_1 u_1(k) + B_2 u_2(k), \quad x(0) = x_0, \quad (1)$$

where $x(k) \in \mathbb{R}^d$ is the state, $u_i(k) \in U_i \subseteq \mathbb{R}^{d_i}$ is the control of player $i \in \{1, 2\}$, and U_i is compact.

Definition 2.2 (State-Feedback Mixed Strategy). *A state-feedback mixed strategy for player i is a measurable map*

$$\alpha_i : \{0, 1, \dots, N-1\} \times \mathbb{R}^d \times \mathbb{R}^{m_i} \rightarrow U_i, \quad (2)$$

such that $u_i(k) = \alpha_i(k, x(k), z_i(k))$, where $\{z_i(k)\}_{k=0}^{N-1}$ are i.i.d. random variables drawn from a fixed latent distribution on a compact set $Z_i \subseteq \mathbb{R}^{m_i}$.

When Z_i is a single point, the mixed strategy degenerates to a *pure strategy* (deterministic given the state). When $Z_i = [0, 1]^{m_i}$ with a uniform distribution, a suitable $\alpha_i(k, x, \cdot)$ can reshape the distribution of $u_i(k)$ to model *any* distribution on U_i .

Definition 2.3 (Payoff Functional). *Player 1 minimizes and Player 2 maximizes*

$$J(\alpha_1, \alpha_2) = \mathbb{E} \left[\sum_{k=0}^{N-1} (\|x(k)\|_Q^2 + \|u_1(k)\|_{R_1}^2 - \|u_2(k)\|_{R_2}^2) + \|x(N)\|_{Q_N}^2 \right], \quad (3)$$

where $Q, Q_N \succeq 0$, $R_1 \succeq 0$, $R_2 \succeq 0$.

Definition 2.4 (Nash Equilibrium). *The game value $V(x_0)$ and equilibrium strategy pair (α_1^*, α_2^*) satisfy*

$$V(x_0) = \inf_{\alpha_1} \sup_{\alpha_2} J(\alpha_1, \alpha_2) = \sup_{\alpha_2} \inf_{\alpha_1} J(\alpha_1, \alpha_2), \quad (4)$$

$$J(\alpha_1, \alpha_2^*) \geq J(\alpha_1^*, \alpha_2^*) \geq J(\alpha_1^*, \alpha_2) \quad \forall \alpha_1, \alpha_2. \quad (5)$$

2.2 One-Dimensional Pursuit-Evasion Game (Benchmark)

We instantiate the general framework with the following concrete example, which serves as our primary benchmark.

Example 2.5 (1D Pursuit-Evasion). *Set $d = d_1 = d_2 = 1$, $A = 1$, $B_1 = B_2 = 1$:*

$$x(k+1) = x(k) + u_1(k) + u_2(k), \quad x(0) = x_0 = 2, \quad (6)$$

with $U_1 = [-c_1, c_1] = [-3, 3]$, $U_2 = [-c_2, c_2] = [-1, 1]$, horizon $N = 5$, and payoff

$$J(\alpha_1, \alpha_2) = \sum_{k=0}^{N-1} (Q x(k)^2 + R_1 u_1(k)^2 - R_2 u_2(k)^2) + Q_N x(N)^2. \quad (7)$$

Parameter settings:

Parameter	Symbol	Current value	Planned value
Initial state	x_0	2	2
Pursuer bound	c_1	3	3
Evader bound	c_2	1	1
Horizon	N	5	5
Running state cost	Q	0.0	0.1
Terminal state cost	Q_N	1.0	1.0
Pursuer control cost	R_1	0.0	0.0
Evader control cost	R_2	0.0	0.0

The “planned value” column reflects the upcoming modification $Q = 0.1$ to enforce a unique equilibrium (see Section 2.4).

2.3 Analytic Equilibrium under Mixed Strategies ($Q = 0$)

Lemma 2.6 (Values under Pure Strategies; Lemma 1 in original paper). *Under pure strategies only, the upper and lower game values do not coincide:*

$$V_-(x) = \begin{cases} 0 & |x| < (c_1 - c_2)N, \\ [|x| + (c_2 - c_1)N]^2 & |x| \geq (c_1 - c_2)N, \end{cases} \quad (8)$$

$$V_+(x) = \begin{cases} c_2^2 & |x| < (c_1 - c_2)(N - 1) + c_1, \\ [|x| + (c_2 - c_1)N]^2 & |x| \geq (c_1 - c_2)(N - 1) + c_1. \end{cases} \quad (9)$$

With $c_1 = 3, c_2 = 1, N = 5, x_0 = 2$: $V_-(2) = 0$ but $V_+(2) = 1$. No NE exists under pure strategies.

Lemma 2.7 (Values under Mixed Strategies; Lemma 2 in original paper). *Under mixed strategies, the upper and lower values coincide:*

$$V(x) = \begin{cases} c_2^2 = 1 & |x| \leq (c_1 - c_2)(N - 1) + c_1 = 11, \\ [|x| + (c_2 - c_1)N]^2 & |x| > 11. \end{cases} \quad (10)$$

An equilibrium mixed strategy pair (α_1^, α_2^*) can be represented as:*

$$\alpha_1^*(k, x, z_1) = \begin{cases} -\text{sign}(x) |x| & |x| \leq c_1, \\ -\text{sign}(x) c_1 & |x| > c_1, \end{cases} \quad (11)$$

$$\alpha_2^*(k, x, z_2) = \begin{cases} -\text{sign}(z_2) c_2 & |x| \leq c_1, \\ \text{sign}(x) c_2 & |x| > c_1, \end{cases} \quad (12)$$

where $z_2 \sim \text{Uniform}[-1, 1]$.

Proof sketch. The key insight is the decomposition:

$$\mathbb{E}[|x + u_1 + u_2|^2 \mid \mathcal{F}_k] = (x + \Gamma_1 + \Gamma_2)^2 + \sigma_1^2 + \sigma_2^2, \quad (13)$$

where $\Gamma_i = \mathbb{E}[u_i \mid \mathcal{F}_k]$, $\sigma_i^2 = \text{Var}[u_i \mid \mathcal{F}_k]$, with the constraint $\sigma_i^2 \leq c_i^2 - \Gamma_i^2$.

Player 1 (minimizer) sets $\Gamma_1 = -x - \Gamma_2$ (if feasible) and $\sigma_1^2 = 0$ to minimize both terms.

Player 2 (maximizer) wants to maximize σ_2^2 while choosing Γ_2 optimally. Setting $\sigma_2^2 = c_2^2 - \Gamma_2^2$ and $\Gamma_2 = 0$ yields $\sigma_2^2 = c_2^2$, and the value becomes c_2^2 whenever $|x| \leq c_1$ (so P1 can fully cancel the mean).

The two-point distribution $u_2 \sim \text{Uniform}\{-c_2, +c_2\}$ achieves exactly $\Gamma_2 = 0$, $\sigma_2^2 = c_2^2$. $\square$

Physical interpretation. At $k = 0$ with $x_0 = 2$:

- Player 1 applies $u_1 = -2$ (a pure feedback law), driving $x(1) = u_2(0) \in \{-1, +1\}$.
- Player 2 applies random $u_2 \in \{-1, +1\}$ with equal probability (mixed strategy), injecting irreducible variance $c_2^2 = 1$.
- At equilibrium: $\mathbb{E}[J] = \mathbb{E}[x(N)]^2 + \text{Var}[x(N)] = 0 + c_2^2 = 1$.

2.4 Non-Uniqueness of Equilibrium Strategies ($Q = 0$)

Proposition 2.8 (Non-uniqueness of α_1^* at $k = 0$). *In Example 2.5 with $Q = 0$, the equilibrium strategy for Player 1 at $k = 0$ is **not unique**. Specifically, for any $u_1(0) \in [-3, 3]$, there exists a complete equilibrium strategy α_1 with $\alpha_1(0, x_0, \cdot) \equiv u_1(0)$ that, paired with the standard α_2^* , forms a Nash equilibrium.*

Proof. We proceed in three steps.

Step 1: Value function flat region. From Lemma 2.7, the value function at $k = 1$ satisfies $V(1, x) = 1$ for all $|x| \leq T(1)$, where

$$T(1) = (c_1 - c_2)(N - 2) + c_1 = 2 \times 3 + 3 = 9.$$

Step 2: All reachable states lie in the flat region. For any $u_1(0) \in [-3, 3]$ and $u_2(0) \in [-1, 1]$:

$$|x(1)| = |2 + u_1(0) + u_2(0)| \leq 2 + 3 + 1 = 6 < 9 = T(1).$$

Therefore $V(1, x(1)) = 1$ regardless of actions at $k = 0$.

Step 3: Bellman equation yields constant. The dynamic programming equation at $k = 0$ reads:

$$V(0, 2) = \sup_{\alpha_2(0)} \inf_{\alpha_1(0)} \mathbb{E}[V(1, 2 + u_1(0) + u_2(0))] = \sup_{\alpha_2(0)} \inf_{\alpha_1(0)} \mathbb{E}[1] = 1.$$

Since the objective is constant in $(\alpha_1(0), \alpha_2(0))$, *every* pair of actions at $k = 0$ is a saddle point of this single-step sub-game.

Step 4: Verification of saddle-point condition. Consider the alternative equilibrium α_1^B with $u_1^B(0) = -1$ and $u_1^B(k, x) = -\text{sign}(x) \min(|x|, c_1)$ for $k \geq 1$, paired with the standard α_2^* .

Left inequality (P2's optimality): For any α_1 , since $|x(1)| \leq 6 < T(1)$, the sub-game from $k = 1$ has value 1. Player 2's strategy α_2^* achieves this maximum, so $J(\alpha_1, \alpha_2^*) \geq 1$.

Right inequality (P1's optimality): Under α_1^B :

$$\begin{aligned} x(1) &= 2 + (-1) + u_2(0) = 1 + u_2(0) \in [0, 2], \\ x(2) &= x(1) + (-x(1)) + u_2(1) = u_2(1) \in [-1, 1], \\ x(k) &= u_2(k-1) \in [-1, 1] \quad \text{for } k = 3, 4, 5. \end{aligned}$$

Thus $J = \mathbb{E}[x(5)^2] = \mathbb{E}[u_2(4)^2] = c_2^2 = 1$. With P2 using the two-point distribution, $J(\alpha_1^B, \alpha_2) \leq 1$ for any α_2 (since P1 can always zero the mean, and no α_2 can achieve variance exceeding c_2^2).

Combining: $J(\alpha_1^B, \alpha_2^*) = 1 = V^*$, confirming the saddle-point condition. $\square$

Remark 2.9 (Root cause and remedy). *The non-uniqueness arises because:*

(a) $Q = 0$: no running cost penalizes intermediate states.

(b) $c_1 \gg c_2$: Player 1 has ample control authority to “catch up” at any future step.

Adding a running cost $Q > 0$ penalizes $|x(k)|^2$ at every step, forcing Player 1 to greedily minimize the state at each time step. This eliminates the flat region in the per-step optimization and restores uniqueness.

2.5 The SDG Approximation Result

The central theoretical contribution of the original paper is to show that the optimization landscape of zero-sum LQ games under mixed strategies can be *dimensionally reduced*.

Definition 2.10 (Conditional Moments). *For player i , define*

$$\Gamma_i(k, x(k)) \triangleq \mathbb{E}[u_i(k) \mid \mathcal{F}_k], \quad (14)$$

$$\Sigma_i(k, x(k)) \triangleq \text{Cov}[u_i(k) \mid \mathcal{F}_k]. \quad (15)$$

The aggregate moments of the combined control input $B_1 u_1 + B_2 u_2$ are:

$$\Gamma(k, x) = B_1 \Gamma_1(k, x) + B_2 \Gamma_2(k, x), \quad (16)$$

$$\Sigma(k, x) = B_1 \Sigma_1(k, x) B_1^\top + B_2 \Sigma_2(k, x) B_2^\top. \quad (17)$$

Theorem 2.11 (SDG Approximation; Theorems 1–2 in original paper). *Given a sampling interval $\delta \in (0, 1)$, there exists an SDE of the form*

$$dY_t = (\bar{A}Y_t + \bar{B}_1 \bar{\Gamma}_1 + \bar{B}_2 \bar{\Gamma}_2)dt + \sqrt{\delta} (\bar{B}_1 \bar{\Sigma}_1 \bar{B}_1^\top + \bar{B}_2 \bar{\Sigma}_2 \bar{B}_2^\top)^{1/2} dW_t \quad (18)$$

that provides a second-order weak approximation to the game dynamics (1). Moreover, the approximation error in both the payoff and the game value is $O(\delta^2)$.

Key Implication for Algorithm Design

The optimization landscape of a zero-sum LQ game under mixed strategies can be cast over the space $\{\Gamma(k, x(k)), \Sigma(k, x(k))\}_{k=0}^{N-1}$. Therefore, **learning the first two conditional moments is sufficient** for finding an approximate NE. The dimension complexity is $O(N \max\{d_1, d_2\}^2)$ rather than infinite-dimensional.

This result directly motivates our Critic design: the Critic receives $(\Gamma_{-i}, \Sigma_{-i})$ as inputs, not the opponent’s full distribution.

3 Model-Free Actor-Critic Framework

3.1 High-Level Architecture

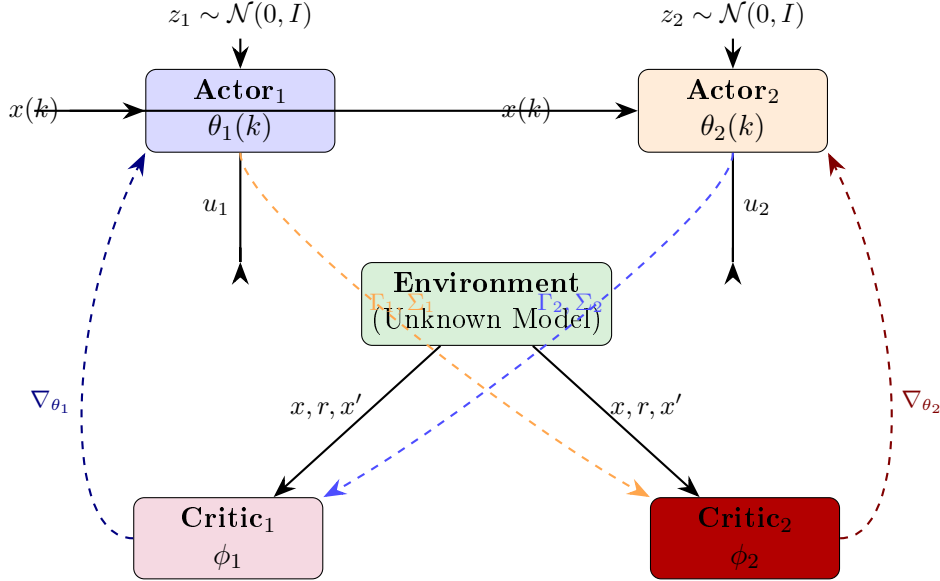


Figure 1: High-level architecture. Solid arrows: data flow. Dashed arrows: moment estimation (stop-gradient) and policy gradient signals.

Comparison with original paper.

Aspect	Original Paper	Our Framework
Model knowledge	Full (A, B_1, B_2) known	Unknown (black-box step)
Gradient source	Backprop through dynamics	Critic-based policy gradient
Network structure	Stacked policy only	Actor + Double Critic per player
Opponent information	Full trajectory	Moments $(\Gamma_{-i}, \Sigma_{-i})$ only
Theoretical basis	Direct minimax	SDG moment reduction + TD learning

3.2 Q-Function Semantics

We define player-specific Q-functions with carefully chosen sign conventions:

Definition 3.1 (Player Q-Functions).

$$Q_1(k, x, u_1) \triangleq \mathbb{E}_\pi[-J \mid x(k) = x, u_1(k) = u_1] \leq 0, \quad (19)$$

$$Q_2(k, x, u_2) \triangleq \mathbb{E}_\pi[+J \mid x(k) = x, u_2(k) = u_2] \geq 0. \quad (20)$$

Under this convention, **both players maximize their own Q-function**:

- Player 1 maximizes $Q_1 = \mathbb{E}[-J] \iff$ minimizes J .
- Player 2 maximizes $Q_2 = \mathbb{E}[+J] \iff$ maximizes J .

The unified Actor loss for both players is therefore:

$$\mathcal{L}_{\text{actor},i} = -\mathbb{E}_{\text{batch}}[Q_i^{\text{cons}}(x, u_i, \Gamma_{-i}, \Sigma_{-i}, k)]. \quad (21)$$

The reward signal from the environment is $r = -(\text{stage cost from P1's perspective})$, so:

$$r_1 = r, \quad r_2 = -r \quad (\text{zero-sum}). \quad (22)$$

4 Network Architectures

4.1 Actor Network

Each player i has an Actor network $\alpha_i(k, x, z; \theta_i)$:

Component	Specification
Input	$[x, z, e(k)] \in \mathbb{R}^{1+m+d_t}$ $x \in \mathbb{R}^1$: state $z \sim \mathcal{N}(0, I_m)$: noise ($m = 4$) $e(k) \in \mathbb{R}^{d_t}$: sinusoidal time embedding ($d_t = 8$)
Hidden	Linear($1+m+d_t, 64$) $\rightarrow$ Mish $\rightarrow$ Linear($64, 64$) $\rightarrow$ Mish
Output	Linear($64, 1$) $\rightarrow \tanh(\cdot) \times c_i$ guarantees $u_i \in [-c_i, c_i]$
Init	Xavier uniform for all weights, zeros for biases

Time embedding. The normalized time $\bar{k} = k/(N-1)$ is encoded via sinusoidal features:

$$e(k)_{2j} = \sin(\bar{k} \cdot 10^{j/d_t}), \quad e(k)_{2j+1} = \cos(\bar{k} \cdot 10^{j/d_t}), \quad j = 0, \dots, d_t/2 - 1. \quad (23)$$

Mish activation. $\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x))$. Smooth, non-monotonic, avoids dying neurons.

Noise input as distribution generator. The noise $z \sim \mathcal{N}(0, I_4)$ allows the Actor to output a state-conditional *distribution* over U_i . For a fixed state x , different noise realizations z produce different actions u_i , thereby defining $\pi_i(\cdot|x)$. This is the direct neural-network analog of the mixed strategy map $\alpha_i(k, x, z)$ from the original paper.

4.2 Critic Network (Double Q)

Each player i has a **Double Q-Critic** consisting of two independent Q-networks:

Component	Specification
Input	$[x, u_i, \Gamma_{-i}, \Sigma_{-i}, e(k)] \in \mathbb{R}^{1+1+1+1+d_t}$ $x \in \mathbb{R}^1$: state $u_i \in \mathbb{R}^1$: own action $\Gamma_{-i} \in \mathbb{R}^1$: opponent's conditional mean $\Sigma_{-i} \in \mathbb{R}^1$: opponent's conditional variance $e(k) \in \mathbb{R}^{d_t}$: time embedding ($d_t = 8$)
Hidden	Linear($4+d_t, 64$) $\rightarrow$ LayerNorm $\rightarrow$ Mish $\rightarrow$ Linear($64, 64$) $\rightarrow$ LayerNorm $\rightarrow$ Mish
Output	Linear($64, 1$) $\rightarrow$ squeeze (no activation)
Init	Xavier uniform, zero biases

Why opponent moments instead of opponent actions? This is the algorithmic manifestation of the SDG result (Theorem 2.11): the optimization landscape depends on the opponent's strategy only through $(\Gamma_{-i}, \Sigma_{-i})$, not the full distribution. Benefits:

- (i) Dimensionality reduction: 2 scalars vs. an entire distribution.
- (ii) Stability: moments change smoothly as the opponent trains, reducing target drift.
- (iii) Theoretical grounding: directly connected to the paper's main theorem.

Moment estimation. Given the opponent’s Actor network, we estimate moments via Monte Carlo:

$$\hat{\Gamma}_{-i}(k, x) = \frac{1}{K} \sum_{j=1}^K u_{-i}^{(j)}, \quad \hat{\Sigma}_{-i}(k, x) = \frac{1}{K-1} \sum_{j=1}^K (u_{-i}^{(j)} - \hat{\Gamma}_{-i})^2, \quad (24)$$

where $u_{-i}^{(j)} = \alpha_{-i}(k, x, z^{(j)}; \theta_{-i})$, $z^{(j)} \sim \mathcal{N}(0, I)$, with $K = 128$ samples. This is computed under `torch.no_grad()` to block gradient flow through the opponent.

Double Q and Conservative Q. The Double Q-Critic outputs two independent estimates (q_1, q_2) :

$$Q_i^{\text{mean}}(x, u, \Gamma_{-i}, \Sigma_{-i}, k) = \frac{q_1 + q_2}{2}, \quad (25)$$

$$Q_i^{\text{cons}}(x, u, \Gamma_{-i}, \Sigma_{-i}, k) = \frac{q_1 + q_2}{2} - \beta |q_1 - q_2|, \quad (26)$$

where $\beta = 0.2$. The mean Q is used for TD targets (unbiased), and the conservative Q is used for Actor updates (pessimistic, preventing exploitation of Critic errors).

5 Training Pipeline

5.1 Overview

Algorithm 1: Model-Free AC Training for Zero-Sum LQ Games

Input: Config $\mathcal{C}$, Environment `env`, initial state x_0
Output: Trained Actor₁, Actor₂

```

// Phase 0: Warmup
1 for  $e = 1$  to  $E_{\text{warm}}$  do
2   | Collect trajectories with random policies;
3   | Push transitions  $(k, x, u_1, u_2, r, x', \text{done})$  into buffer  $\mathcal{B}$ ;
4 end
5 for  $s = 1$  to  $S_{\text{warm}}$  do
6   | Update Critic1, Critic2 via TD learning;
7   | Soft-update target networks;
8 end

// Phase 1: Adversarial AC Training
9 for  $e = 1$  to  $E_{\text{total}}$  do
10  | Collect trajectories with current Actor1, Actor2;
11  | for  $s = 1$  to  $S_{\text{critic}}$  do
12    | Update Critic1 (Alg. 2);
13    | Update Critic2 (Alg. 2);
14  | end
15  | Soft-update target networks:  $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-$ ;
16  | for  $s = 1$  to  $S_{\text{actor}}$  do
17    | Update Actor1 (Alg. 3, no entropy);
18    | Update Actor2 (Alg. 3, with entropy);
19  | end
20  | Evaluate and record metrics;
21  | if best model criteria met then
22    | Save checkpoint;
23  | end
24 end

```

5.2 Replay Buffer

The replay buffer $\mathcal{B}$ stores transitions of the form:

$$(k, x, u_1, u_2, r, x', \text{done}) \in \{0, \dots, N-1\} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \mathbb{R} \times \{0, 1\}.$$

Key properties:

- **Capacity:** 100,000 transitions with circular overwrite.
- **Mixed sampling:** Each batch of size B is composed of $\lfloor 0.5B \rfloor$ recent transitions and $\lfloor 0.5B \rfloor$ uniformly sampled historical transitions. This balances freshness (current policy distribution) and coverage (past state-action diversity).
- **Time-step field:** Stored explicitly so that time-dependent quantities (embeddings, moments, done flags) can be correctly reconstructed during training.

5.3 Critic Update

Algorithm 2: Critic Update for Player i

```

Input: Batch  $\{(k, x, u_1, u_2, r, x', \text{done})\}$  from  $\mathcal{B}$ 
1  $r_{\text{self}} \leftarrow r$  if  $i = 1$ , else  $-r$ ; // zero-sum sign
   // Compute TD target using target network
2 for each unique  $k' = k + 1$  in the batch where  $\text{done} = 0$  do
3    $\hat{\Gamma}_{-i}, \hat{\Sigma}_{-i} \leftarrow \text{estimate\_moments}(\text{Actor}_{-i}, x', k')$ ;
4    $u'_i \leftarrow \text{Actor}_i.\text{sample}(x', k')$ ;
5 end
6  $q_{\text{next}} \leftarrow Q_i^{\text{mean}, \text{target}}(x', u'_i, \hat{\Gamma}_{-i}, \hat{\Sigma}_{-i}, k')$ ;
7  $y \leftarrow r_{\text{self}} + (1 - \text{done}) \cdot q_{\text{next}}$ ; // bootstrap only non-terminal
   // Current prediction
8  $\hat{\Gamma}_{-i}, \hat{\Sigma}_{-i} \leftarrow \text{estimate\_moments}(\text{Actor}_{-i}, x, k)$ ;
9  $\hat{q}_1, \hat{q}_2 \leftarrow \text{Critic}_i(x, u_{\text{self}}, \hat{\Gamma}_{-i}, \hat{\Sigma}_{-i}, e(k))$ ;
10  $\mathcal{L}_{\text{critic}} \leftarrow \text{Huber}(\hat{q}_1, y) + \text{Huber}(\hat{q}_2, y)$ ;
11 Update  $\phi_i$  via Adam with gradient clipping (max norm 5.0);

```

Key design choices.

- (a) **Mean Q for targets:** Using Q^{mean} (Eq. 25) for TD targets avoids the negative bias that would result from using Q^{cons} .
- (b) **Target networks:** Separate target networks ϕ_i^- with soft-update $\tau = 0.005$ stabilize the moving TD target.
- (c) **Skip done samples:** When $\text{done} = 1$, the target is simply r_{self} with no bootstrap—the $(1 - \text{done})$ factor zeroes out q_{next} .
- (d) **Time embedding cache:** Embeddings are computed once per unique k value (at most N calls) rather than per sample (B calls), yielding a significant speedup.
- (e) **Moment estimation under stop-gradient:** $\hat{\Gamma}_{-i}$ and $\hat{\Sigma}_{-i}$ are computed inside `torch.no_grad()`, preventing gradient leakage to the opponent.

5.4 Actor Update

Algorithm 3: Actor Update for Player i

Input: Batch $\{(k, x)\}$ from $\mathcal{B}$, Critic $_i$ (frozen)

```

1 Freeze Critic $_i$  parameters;
   // Opponent moments (stop-gradient)
2 for each unique  $k$  in batch do
3   |  $\hat{\Gamma}_{-i}, \hat{\Sigma}_{-i} \leftarrow \text{estimate\_moments}(\text{Actor}_{-i}, x, k)$ ;
4 end

   // Forward pass with gradient through Actor
5 for each unique  $k$  in batch do
6   |  $z \sim \mathcal{N}(0, I_m)$ ;
7   |  $u_i \leftarrow \text{Actor}_i(x, z, e(k); \theta_i)$ ;           // gradient flows through  $\theta_i$ 
8 end
9 Reassemble  $u_i$  in original batch order via inverse index mapping;
10  $q \leftarrow Q_i^{\text{cons}}(x, u_i, \hat{\Gamma}_{-i}, \hat{\Sigma}_{-i}, e(k))$ ;
11  $\mathcal{L}_q \leftarrow -\text{mean}(q)$ ;                          // maximize Q for both players
   // Entropy regularization (Player 2 only)
12 if  $i = 2$  and  $\lambda_H > 0$  then
13   | Sample  $n_s = 8$  actions per state via Actor $_2$ ;
14   |  $\hat{\sigma}^2 \leftarrow \text{mean of per-state variances}$ ;
15   |  $\hat{H} \leftarrow \frac{1}{2} \log(2\pi e \hat{\sigma}^2 + \epsilon)$ ;
16   |  $\mathcal{L} \leftarrow \mathcal{L}_q - \lambda_H \cdot \hat{H}$ ;
17 end
18 else
19   |  $\mathcal{L} \leftarrow \mathcal{L}_q$ ;
20 end
21 Unfreeze Critic $_i$ ;
22 Update  $\theta_i$  via Adam with gradient clipping (max norm 2.0);
```

Why torch.cat with inverse-index instead of in-place assignment? A critical implementation detail: the Actor forward pass is computed per- k -group and reassembled via `torch.cat` followed by inverse-index reordering. This preserves the computation graph for backpropagation. An alternative approach using in-place tensor assignment (e.g., `u_self[mask] = u_sub`) would **silently detach gradients** in PyTorch, breaking the Actor’s learning signal.

Entropy regularization for Player 2. Player 2’s equilibrium strategy is a mixed strategy (the two-point distribution $\{-c_2, +c_2\}$). Without entropy regularization, the Actor network may collapse to a pure strategy (a known failure mode in minimax training). The differential entropy approximation

$$\hat{H}(\pi_2|x) \approx \frac{1}{2} \log(2\pi e \hat{\sigma}^2) \quad (27)$$

is the entropy of a Gaussian with variance $\hat{\sigma}^2$, serving as a smooth lower bound that encourages the Actor to maintain distributional spread. The coefficient $\lambda_H = 0.3$ balances exploration (high entropy) against exploitation (high Q).

Player 1’s equilibrium is a pure strategy, so entropy regularization is *not* applied to Actor $_1$ —doing so would counterproductively prevent convergence to a deterministic policy.

5.5 Evaluation

At regular intervals during training, the following metrics are computed over $n = 2000$ Monte Carlo trajectories starting from x_0 :

Metric	Definition	Target
ValErr	$ E[J] - V^* $	< 0.1
$\bar{u}_1(0)$	$\mathbb{E}[u_1(0) \mid x_0]$	≈ -2.0 (with $Q > 0$)
$\sigma[u_1(0)]$	$\text{Std}[u_1(0) \mid x_0]$	≈ 0 (pure)
$\bar{u}_2(0)$	$\mathbb{E}[u_2(0) \mid x_0]$	≈ 0 (symmetric)
$\sigma[u_2(0)]$	$\text{Std}[u_2(0) \mid x_0]$	≈ 1.0 (mixed)

5.6 Hyperparameter Summary

Hyperparameter	Symbol	Value
Noise dimension	m	4
Time embedding dimension	d_t	8
Hidden layer width	—	64
Actor learning rate	η_a	5×10^{-5}
Critic learning rate	η_c	3×10^{-4}
Critic updates per episode	S_{critic}	20
Actor updates per episode	S_{actor}	2
Soft-update coefficient	τ	0.005
Conservative Q penalty	β	0.2
Entropy coefficient (P2)	λ_H	0.3
MC samples for moments	K	128
Batch size (Critic/Actor)	B	512
Training batch size	—	32
Replay buffer capacity	—	100,000
Recent sampling ratio	—	0.5
Warmup episodes	E_{warm}	200
Warmup Critic steps	S_{warm}	500
Total training episodes	E_{total}	5000
Best-model P1 constraint	—	$\bar{u}_1(0) < -0.5$

6 Current Results and Observations

6.1 Training Trajectory ($Q = 0$, 5000 episodes)

The training was conducted with the $Q = 0$ configuration. Key observations from the training log:

Ep	ValErr	J	$\bar{u}_1(0)$	$\sigma[u_1(0)]$	$\bar{u}_2(0)$	$\sigma[u_2(0)]$
200	0.626	0.374	-1.22	0.269	+0.31	0.525
1000	0.129	0.871	-1.95	0.164	-0.90	0.347
2200	0.001	0.999	-0.79	0.183	+0.19	0.922
2833	0.000	1.000	-1.07	0.160	-0.10	0.955
5000	0.022	1.022	-0.98	0.037	-0.01	0.984

Best model was restored from episode 2833 with ValErr ≈ 0 .

Success.

- Game value converges accurately: $\text{ValErr} < 0.01$.
- Player 2 learns the mixed strategy: $\bar{u}_2 \approx 0$, $\sigma[u_2] \approx 0.955 \approx c_2$.
- CDF analysis confirms P2’s output closely approximates the two-point distribution.

Expected observation: P1 converges to $\bar{u}_1 \approx -1$ instead of -2 . This is *not* a bug. As proven in Proposition 2.8, the equilibrium strategy for Player 1 is non-unique when $Q = 0$. Any $u_1(0) \in [-3, 3]$ can form part of an equilibrium, because all reachable states at $k = 1$ lie within the flat region of the value function $V(1, \cdot)$. The algorithm converges to a valid but non-myopic equilibrium.

6.2 Planned Fix: Adding Running Cost $Q > 0$

Setting $Q = 0.1$ introduces a per-step state penalty $Q|x(k)|^2$, which:

- Breaks the flat region in the per-step Bellman equation: $V(k, x)$ becomes strictly convex in x for every k .
- Forces Player 1 to greedily minimize $|x(k)|$ at each step, restoring uniqueness of the equilibrium.
- Provides stronger learning signal for the Critic, since running costs contribute to every non-terminal transition’s reward.

Remark 6.1. *With $Q = 0.1$, the analytic equilibrium changes. While the exact solution would need to be recomputed (a new Riccati-like recursion), the qualitative behavior should be: Player 1 applies strong corrective control at every step, and Player 2 maintains maximum variance. The game value will be larger than $c_2^2 = 1$ due to the accumulated running costs.*

7 Summary of Key Bug Fixes and Design Decisions

During development, several critical issues were identified and resolved:

#	Issue	Resolution
1	Reward sign	P2’s TD target uses $r_{\text{self}} = -r$ (zero-sum).
2	TD target bias	Use Q^{mean} (not Q^{cons}) for TD target, preventing systematic negative bias.
3	Gradient disconnection	Actor uses <code>torch.cat</code> + inverse index to reassemble actions (not in-place <code>tensor[mask] = val</code>).
4	Actor loss direction	Both players use $-Q.\text{mean}()$ as loss, since Q -functions are defined with player-specific signs.
5	Initial state noise	Removed x_0 noise. Fixed $x_0 = 2$ prevents P1 from learning symmetric-flip artifacts.
6	Best model selection	Added P1 direction constraint ($\bar{u}_1(0) < -0.5$) to prevent saving models where P1 goes in the wrong direction.

8 Next Steps

- Add running cost $Q = 0.1$:** Rerun training to enforce unique equilibrium and verify $\bar{u}_1(0) \rightarrow -x_0$. This requires recomputing the analytic benchmark.
- Tighter Critic targets:** Consider adding TD target clipping with $\max_q = (|x_0| + N(c_1 + c_2))^2$ to prevent early-stage divergence.
- Multi-dimensional extension:** Scale to $d > 1$ with matrix-valued controls and costs, testing whether the $O(N \max\{d_1, d_2\}^2)$ complexity prediction holds in practice.

4. **Comparison with model-based baseline:** Implement the original paper's stacked network with known dynamics and compare sample efficiency, convergence speed, and final quality.